

TTTGameData

Version Control Document - DB Lab

Group Name	TTTGameData
Project	Super Tic-Tac-Toe Multiplayer Database
Members listed in source doc	Muhammad Hammad Shakeel Sikandar Khaleel Aurshia Mudassar
GitHub Repository Link	https://github.com/hammadshakeelai/Game-Database-Project

Version Control Log

Milestone	Date	Version	Remarks
ERD + Schema	24 / 4 / 2026	V1.0	Initial schema; improvement required
Schema Improvement	29 / 4 / 2026	V1.1	M2 preparation
M2: 3NF	5 / 5 / 2026	V2.0	Normalized ERD and table justification
M3: CSVs	12 / 5 / 2026	V3.0	Clean synthetic dataset and dataflow
M4: DDL	15 / 5 / 2026	V3.0	DDL scripts and EER verification
M5: DML	17 / 5 / 2026	V4.0	Data population and validation queries

Project Overview

The database supports a Super Tic-Tac-Toe multiplayer platform with ranked games, tournaments, friends, groups, chat, notifications, and leaderboard features. Core entities include PLAYER, GAMETYPE, TOURNAMENT, GAME, GAME_HISTORY, GROUP_T, GROUP_LIST, CHAT, and MESSAGE. Junction tables are used for many-to-many relationships, and foreign keys maintain referential integrity across gameplay and social features.

Updated ERD Diagram

Final normalized ERD used for the DB Lab submission. The design is normalized up to 3NF and includes lookup, core, event, bridge, log, communication, notification, and leaderboard tables.

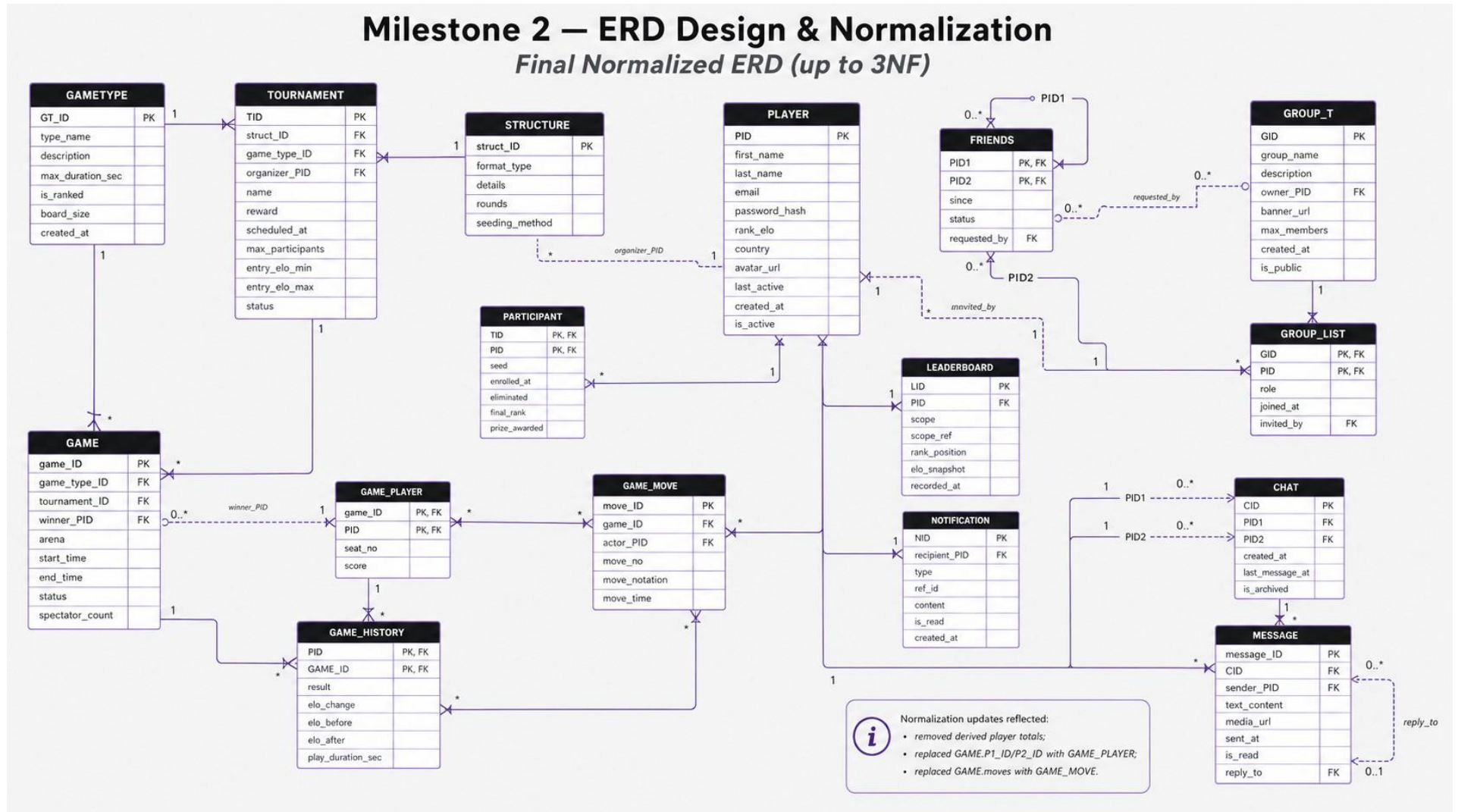


Figure 1. Final normalized ERD for the Super Tic-Tac-Toe database.

Normalization Walkthrough

The final ERD is treated as the source of truth. The goal of normalization is to reduce redundancy, prevent update anomalies, and keep each table focused on one entity or relationship.

Normal Form	Justification and changes made
1NF	All fields are atomic. Repeating groups and multi-valued fields were separated into child tables. In particular, the previous GAME.moves list was replaced with GAME_MOVE, and message details are stored as individual MESSAGE rows.
2NF	Composite-key tables store only attributes that depend on the full key. PARTICIPANT(TID, PID) stores enrollment details, GAME_PLAYER(game_ID, PID) stores seat/score details, GAME_HISTORY(PID, GAME_ID) stores game outcome details, GROUP_LIST(GID, PID) stores membership details, and FRIENDS(PID1, PID2) stores relationship details.
3NF	Non-key attributes depend only on the key. Derived player totals were removed from PLAYER because they can be calculated from GAME_HISTORY. Reusable game variant rules are stored in GAMETYPE, tournament formats are stored in STRUCTURE, and profile details are not duplicated in relationship tables.

Important 3NF Changes Reflected in the ERD

- Removed derived player totals from PLAYER so wins/losses/draws can be calculated from GAME_HISTORY.
- Replaced fixed GAME.P1_ID and GAME.P2_ID columns with GAME_PLAYER for flexible participation.
- Replaced the multi-valued GAME.moves attribute with GAME_MOVE for one row per atomic move.
- Separated GROUP_T from GROUP_LIST and TOURNAMENT from PARTICIPANT to keep membership/enrollment attributes out of parent tables.

Main Table Groups

Category	Tables	Reason
Reference / lookup	GAMETYPE, STRUCTURE	Reusable rule and format definitions without repeating them in event rows.
Core actor	PLAYER	One profile row per account, referenced by gameplay, social, and communication features.
Event records	TOURNAMENT, GAME	Stores scheduled competitions and individual match instances.
Associative tables	PARTICIPANT, GAME_PLAYER, GAME_HISTORY, FRIENDS, GROUP_LIST	Resolves many-to-many and self-referencing relationships while storing relationship-specific attributes.
Detail / log tables	GAME_MOVE, MESSAGE, NOTIFICATION, LEADERBOARD, CHAT, GROUP_T	Stores detailed activity, communication, group metadata, and snapshots separately.

Dataflow Description

Data enters through registration, game-type setup, tournament creation, gameplay activity, groups, friendships, chat messages, and notifications. The normalized flow loads parent/reference data before dependent child and bridge tables.

1. Reference and user data enters first

PLAYER stores registered users. GAMETYPE stores available game modes. STRUCTURE stores tournament formats such as single elimination, round robin, Swiss, and double elimination.

2. Tournament data depends on parent tables

TOURNAMENT references GAMETYPE, STRUCTURE, and the organizer PLAYER. PARTICIPANT then links players to tournaments with the composite key TID + PID and stores enrollment state.

3. Gameplay data flows into game tables

GAME records match instances. GAME_PLAYER stores the players in each game, GAME_MOVE stores the move sequence, and GAME_HISTORY stores each player result and Elo change after completion.

4. Social data flows through groups and friendships

GROUP_T stores group metadata and owner. GROUP_LIST connects players to groups and stores roles/join dates. FRIENDS stores player-to-player relationship records with requested_by and status.

5. Communication and notifications

CHAT stores a conversation between two players. MESSAGE stores each message, sender, optional media, read state, and reply reference. NOTIFICATION stores system messages caused by game results, friend requests, group invites, tournament invites, and rank updates.

6. Database outputs

Tournament standings, player match history, ranking reports, social/group reports, chat feeds, notification feeds, and future AI/recommendation inputs can be produced by joining the normalized tables.

Recommended Load Order

player -> gametype -> structure -> tournament -> participant -> game -> game_player -> game_move -> game_history -> group_t -> group_list -> friends -> leaderboard -> notification -> chat -> message

GitHub Repository and Submission Checklist

GitHub Repository Link	https://github.com/hammadshakeelai/Game-Database-Project
Filename	TTTGameData_Version_Control_Doc_DBLab.pdf

Requirement	Included in this PDF
Group name and member names	Yes - cover page
Updated ERD diagram	Yes - final normalized ERD page
Normalization walkthrough	Yes - 1NF, 2NF, and 3NF with justifications
Dataflow description	Yes - reference/user, tournament, gameplay, social, communication, outputs
GitHub repository link	Yes - https://github.com/hammadshakeelai/Game-Database-Project

Validation Notes

- The schema contains 16 core tables matching the final normalized ERD.
- Primary keys, composite keys, foreign keys, uniqueness rules, and check constraints are used to preserve integrity.
- After the DML update/delete demo, validation queries should confirm no key-column nulls and no orphaned foreign-key references.